



VITYarthi- Computer Vision Evaluated Project

COMPUTER VISION IMAGE ANALYZER

An Image Analysis System Using OpenCV

EVALUATED PROJECT REPORT

Submitted by-

Name- Sneha Gayatri Vogeti

Reg. no.- 24BAC10029

Subject- Computer Vision (CSE3010)

Professor- Dr. Neha Rathore

Slot- C11 + C12

Academic Year: 2026–27

TABLE OF CONTENTS

1. INTRODUCTION	4
2. PROBLEM STATEMENT	5
3. OBJECTIVES	6
4. REQUIREMENTS	7
4.1 Functional Requirements	7
4.2 Non-Functional Requirements	8
4.3 Hardware Requirements	8
4.4 Software Requirements	9
5. SYSTEM DESIGN	9
5.1 System Architecture	9
5.2 Workflow	10
5.3 Module Description	11
6. DESIGN RATIONALE	12
7. IMPLEMENTATION	13
8. RESULTS AND SCREENSHOTS	14
9. TESTING	17

10. CHALLENGES AND LEARNINGS	19
11. FUTURE ENHANCEMENTS	20
12. CONCLUSION	21
13. REFERENCES	22

1. INTRODUCTION

Computer vision is a field of artificial intelligence that enables computers to obtain meaningful information from digital images and videos. Image analysis is an important application of computer vision because images can contain useful information about their dimensions, visual quality, structural features, and objects or regions of interest.

The Computer Vision Image Analyzer is a Python-based application developed using the OpenCV library. The system provides a simple automated workflow for analysing a user-provided image. It performs face detection, basic image property analysis, image quality assessment, and edge detection.

The application accepts a supported image as input and processes it through multiple independent analysis modules. The results are presented through a processed image, an edge-detection image, console output, and a structured text report. Input validation and error handling are also incorporated to make the system more reliable.

The project demonstrates the practical application of fundamental computer vision and image-processing concepts in a modular software system. It is designed primarily as an educational project for understanding how different image-processing operations can be combined into a single automated analysis workflow.

2. PROBLEM STATEMENT

Digital images contain useful visual information, but extracting even basic information from an image manually can be time-consuming and inconsistent. Tasks such as identifying faces, determining image properties, evaluating basic image quality, and identifying structural features require separate image-processing operations.

Therefore, there is a need for a simple and modular computer vision application that can accept an image as input, perform multiple fundamental image-analysis operations, and present the results in an understandable form.

The proposed Computer Vision Image Analyzer addresses this problem by integrating face detection, image property analysis, image quality assessment, and edge detection into a single automated workflow. The system also validates input files, handles processing errors, saves visual outputs, and generates a structured analysis report.

3. OBJECTIVES

The main objectives of the Computer Vision Image Analyzer are:

1. To develop a modular computer vision application using Python and OpenCV.
2. To accept and validate user-provided image files in supported formats.
3. To detect faces present in an input image using a Haar Cascade classifier.
4. To determine basic image properties such as width, height, number of channels, total pixels, and average brightness.
5. To assess basic image quality using brightness and sharpness measurements.
6. To apply Canny edge detection to identify significant structural boundaries within an image.
7. To generate processed visual outputs and a structured text-based analysis report.
8. To incorporate error handling and automated testing to improve the reliability of the application.
9. To demonstrate the practical application of fundamental computer vision and image-processing techniques in a single workflow.

4. REQUIREMENTS

4.1 *Functional Requirements*

The system shall satisfy the following functional requirements:

1. **Image Input and Validation:** The system shall accept an image file provided by the user and verify that the file exists and has a supported image format.
2. **Image Loading:** The system shall load a valid image into memory for further processing.
3. **Face Detection:** The system shall detect faces present in the input image using a Haar Cascade classifier.
4. **Image Property Analysis:** The system shall determine the image width, height, number of channels, total number of pixels, and average brightness.
5. **Image Quality Assessment:** The system shall evaluate basic image quality using brightness and sharpness measurements.
6. **Edge Detection:** The system shall generate an edge map using the Canny edge-detection algorithm.
7. **Visual Output Generation:** The system shall save an output image containing detected face regions and a separate edge-detection image.
8. **Report Generation:** The system shall generate a text-based report containing the analysis results.
9. **Error Handling:** The system shall display meaningful error messages when an invalid or unreadable input is provided.

4.2 *Non-Functional Requirements*

The system shall satisfy the following non-functional requirements:

1. **Usability:** The application shall provide a simple command-line interface that allows users to run the analysis with minimal configuration.
2. **Reliability:** The system shall validate input files and handle common processing errors without unexpectedly terminating.
3. **Maintainability:** The application shall use separate modules for image loading, validation, face detection, image analysis, quality analysis, edge detection, reporting, and utility operations.
4. **Performance:** The system shall process an input image through the analysis pipeline without unnecessary repeated operations.
5. **Error Handling:** Invalid file paths, unsupported formats, unreadable images, and unavailable detection models shall generate meaningful error messages.
6. **Modularity:** Individual analysis functions shall operate as independent components so that they can be tested and maintained separately.

4.3 *Hardware Requirements*

The application does not require specialized hardware. The minimum practical hardware requirements are:

- A computer or laptop with a modern processor.
- Minimum 4 GB RAM.
- Sufficient storage for the Python environment, source code, input images, and generated outputs.
- Standard display and keyboard for command-line interaction.

No dedicated GPU is required for the current implementation.

4.4 *Software Requirements*

The software requirements for the project are:

- **Operating System:** Windows, Linux, or macOS
- **Programming Language:** Python 3.x
- **Computer Vision Library:** OpenCV
- **Numerical Computing Library:** NumPy
- **Testing Framework:** Pytest
- **Development Environment:** Visual Studio Code or any compatible Python IDE
- **Version Control:** Git and GitHub

5. SYSTEM DESIGN

5.1 *System Architecture*

The Computer Vision Image Analyzer follows a modular processing architecture. The system receives an image from the user and passes it through input validation and image loading before performing multiple independent computer vision and image-analysis operations.

The major components of the architecture are:

- **Input Layer:** Receives the image path provided by the user.
- **Validation Module:** Verifies that the input file exists and has a supported image format.
- **Image Loading Module:** Reads the validated image using OpenCV.
- **Face Detection Module:** Converts the image to grayscale and applies the Haar Cascade classifier to detect faces.
- **Image Analysis Module:** Extracts image dimensions, channel information, total pixels, and average brightness.
- **Quality Analysis Module:** evaluates brightness and calculates a sharpness score using the variance of the Laplacian.

- **Edge Detection Module:** applies grayscale conversion, Gaussian blurring, and Canny edge detection.
- **Output and Reporting Layer:** Saves the processed images and generates a structured text report.

The modular architecture separates individual responsibilities, making the application easier to test, maintain, and extend.

5.2 Workflow

The overall workflow of the system is as follows:

1. The user provides an image path or uses the default test image.
2. The input validation module checks the file path and image format.
3. The image loading module reads the image using OpenCV.
4. The face detection module processes the image and identifies detected faces.
5. The image analysis module calculates basic image properties.
6. The quality analysis module evaluates brightness and sharpness.
7. The edge detection module generates an edge map using the Canny algorithm.
8. The detected face regions are drawn on a copy of the original image.
9. The processed image and edge image are saved to the output directory.
10. The reporting module generates a text report containing the analysis results.
11. The main program displays the key results to the user through the command line.

5.3 Module Description

The project is divided into multiple modules, with each module responsible for a specific operation:

Module	Responsibility
main.py	Controls the overall application workflow
input_validator.py	Validates image paths and supported formats
image_loader.py	Loads images using OpenCV
face_detector.py	Performs face detection
image_analyzer.py	Calculates basic image properties
quality_analyzer.py	Evaluates brightness and sharpness
edge_detector.py	Performs Canny edge detection
image_utils.py	Draws detections and saves images
report_generator.py	Generates the analysis report
test_analyzers.py	Contains automated tests

6. DESIGN RATIONALE

The design of the Computer Vision Image Analyzer was selected to provide a clear separation between input handling, image processing, analysis, and output generation.

OpenCV was selected as the primary computer vision library because it provides established image-processing and computer vision operations required by the project, including image loading, grayscale conversion, face detection, Gaussian filtering, and Canny edge detection.

Haar Cascade face detection was selected because it provides a lightweight classical computer vision approach suitable for a CPU-based educational application. It allows the project to demonstrate an actual detection pipeline without requiring specialized hardware or a large machine-learning model.

Grayscale conversion is used where color information is not necessary for the analysis. It simplifies processing while retaining useful intensity information.

Gaussian blurring is applied before Canny edge detection to reduce high-frequency noise and improve the quality of the resulting edge map.

Laplacian variance is used as a basic sharpness measure because stronger local intensity changes generally produce a higher variance, providing a simple numerical indication of image sharpness.

The application uses a modular architecture rather than placing all operations in a single program file. This improves maintainability and allows individual components to be tested independently.

Finally, the system generates both visual and textual outputs so that users can inspect the processed images while also receiving measurable analysis results in a structured report.

7. IMPLEMENTATION

The application was implemented in Python using OpenCV and NumPy. The implementation is divided into independent modules, with `main.py` coordinating the complete processing workflow.

The input validation component checks whether the specified image exists and whether its extension belongs to the supported formats: JPG, JPEG, PNG, or BMP. A valid image is then loaded using OpenCV.

For face detection, the input image is converted to grayscale and processed using OpenCV's Haar Cascade frontal-face classifier. Detected face coordinates are returned to the main workflow and drawn as rectangular regions on a copy of the original image.

The image analysis module extracts the image dimensions, number of channels, total number of pixels, and average brightness. The quality analysis module uses the average grayscale brightness to classify the image as too dark, too bright, or good. It also calculates a sharpness score using the variance of the Laplacian and classifies the image as blurry or sharp.

The edge detection module converts the image to grayscale, applies Gaussian smoothing, and then performs Canny edge detection. The resulting edge map is saved as a separate output image.

After processing, the system saves the detected-face image, edge image, and text report in the output directory. Exceptions such as missing files, unsupported formats, unreadable images, and unavailable detection models are handled with meaningful error messages.

8. RESULTS

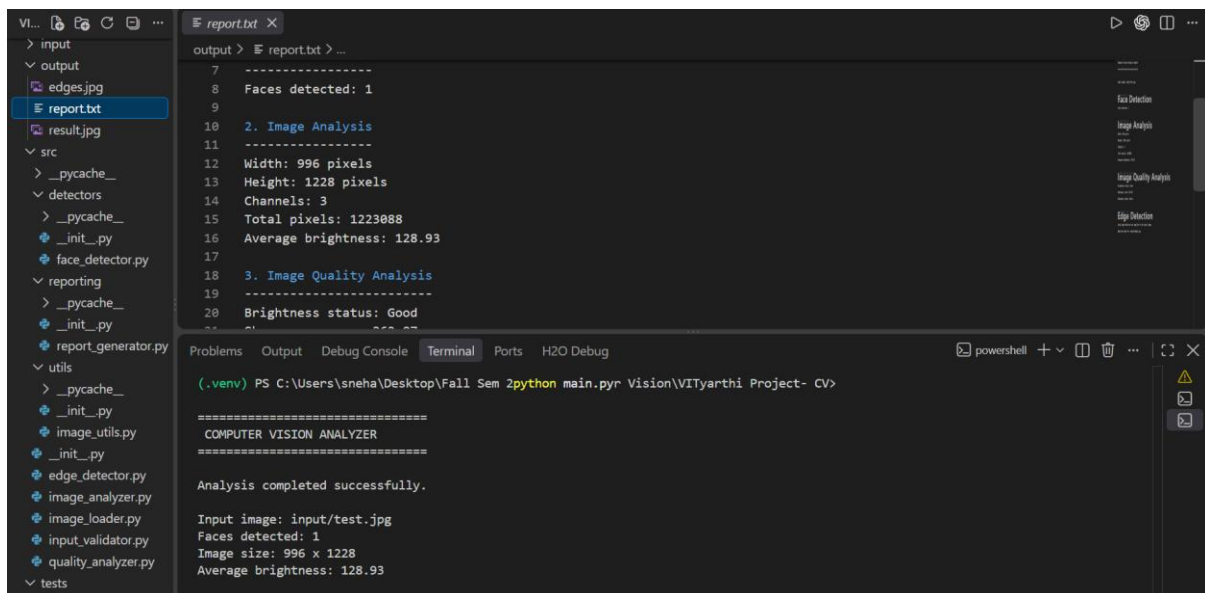
The developed system was executed using a sample input image to verify the complete image-analysis workflow. The application successfully performed face detection, image property analysis, image quality assessment, and edge detection.

For the test image, the system detected 1 face. The image dimensions were 996×1228 pixels, with an average brightness of 128.93. Based on the implemented thresholds, the image was classified as having good brightness and Sharp image quality.

The system generated three primary outputs:

1. result.jpg – the input image with detected face regions highlighted.
2. edges.jpg – the edge map generated using Canny edge detection.
3. report.txt – a structured text report containing the analysis results.

The successful execution demonstrates that the complete processing pipeline operates as intended.



```
report.txt
output > report.txt > ...
7 -----
8 Faces detected: 1
9
10 2. Image Analysis
11 -----
12 Width: 996 pixels
13 Height: 1228 pixels
14 Channels: 3
15 Total pixels: 1223088
16 Average brightness: 128.93
17
18 3. Image Quality Analysis
19 -----
20 Brightness status: Good
21 -----
22
```

```
(.venv) PS C:\Users\sneha\Desktop\Fall Sem 2\python main.py: Vision\VITYarthi Project- CV>
=====
COMPUTER VISION ANALYZER
=====
Analysis completed successfully.

Input image: input/test.jpg
Faces detected: 1
Image size: 996 x 1228
Average brightness: 128.93
```

Figure 1. Successful execution of the Computer Vision Image Analyzer

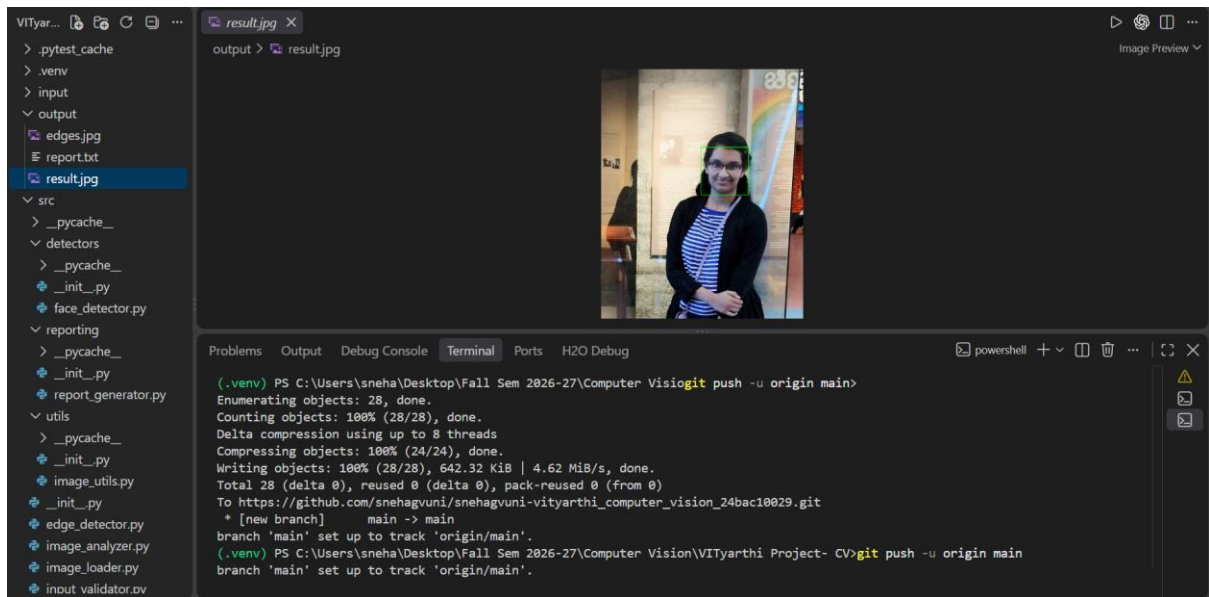


Figure 2. Face detection result

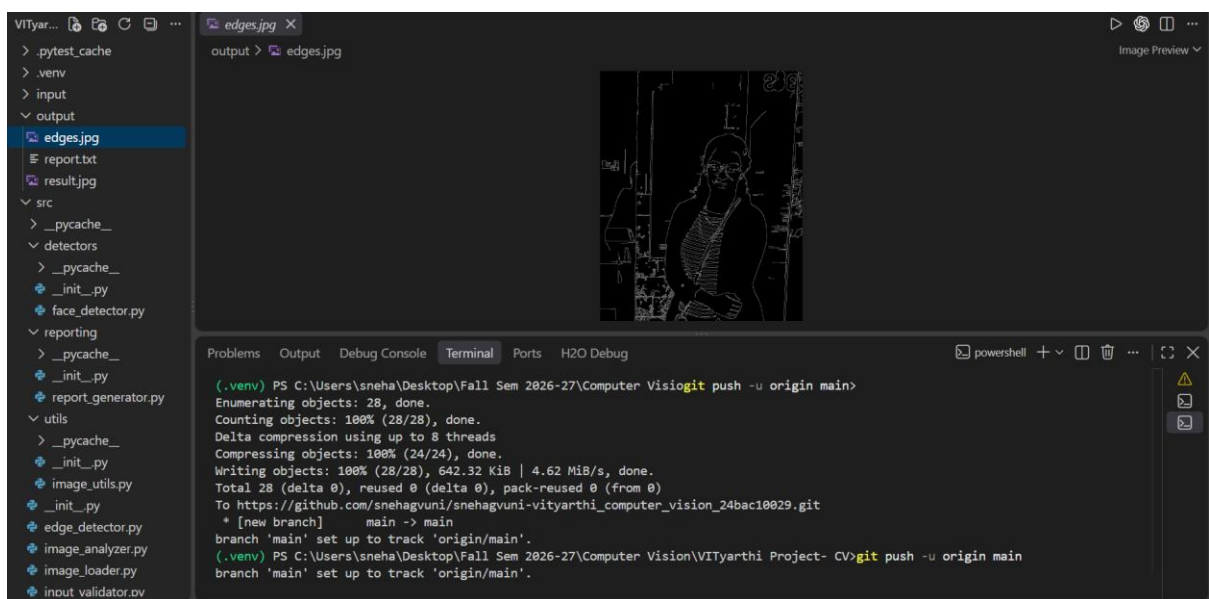
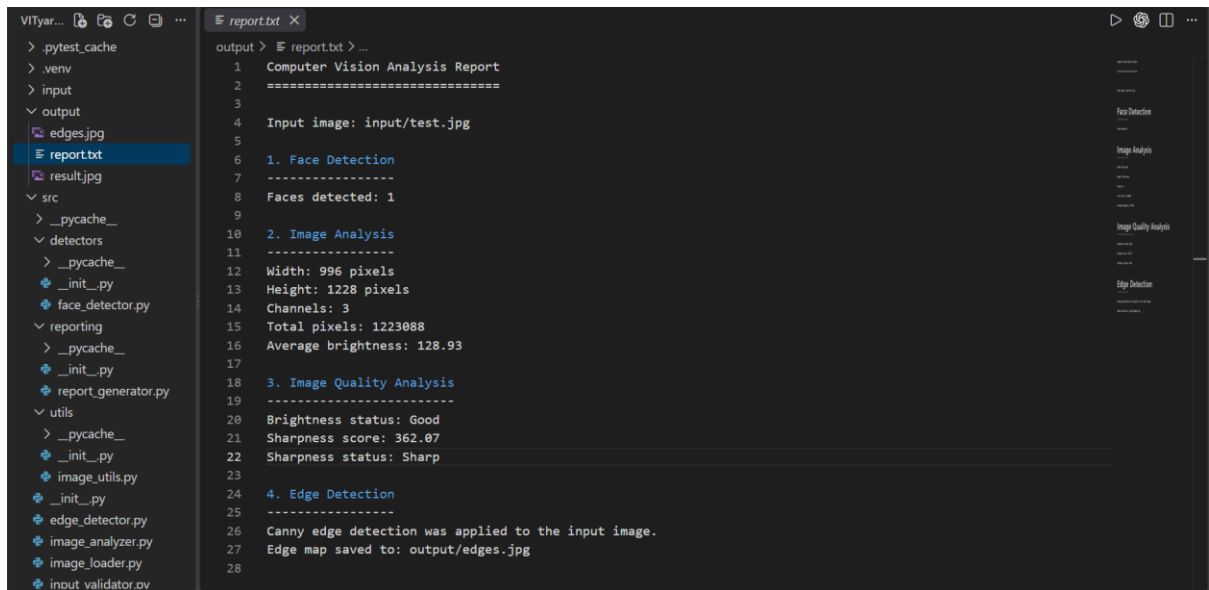


Figure 3. Canny edge detection result



The screenshot shows a code editor with a file explorer on the left and a text editor in the center. The file explorer lists various files and folders, including .pytest_cache, .venv, input, output, edges.jpg, report.txt (selected), result.jpg, src, __pycache__, detectors, __pycache__, __init__.py, face_detector.py, reporting, __pycache__, __init__.py, report_generator.py, utils, __pycache__, __init__.py, image_utils.py, __init__.py, edge_detector.py, image_analyzer.py, image_loader.py, and input_validator.py. The text editor displays the content of report.txt, which is a Computer Vision Analysis Report. The report includes sections for Face Detection, Image Analysis, Image Quality Analysis, and Edge Detection. The Face Detection section reports 1 face detected. The Image Analysis section reports a width of 996 pixels, height of 1228 pixels, 3 channels, and a total of 1223088 pixels. The Image Quality Analysis section reports a brightness status of Good, a sharpness score of 362.07, and a sharpness status of Sharp. The Edge Detection section reports that Canny edge detection was applied to the input image and that the edge map was saved to output/edges.jpg.

```
output > report.txt > ...
1  Computer Vision Analysis Report
2  =====
3
4  Input image: input/test.jpg
5
6  1. Face Detection
7  -----
8  Faces detected: 1
9
10 2. Image Analysis
11 -----
12 Width: 996 pixels
13 Height: 1228 pixels
14 Channels: 3
15 Total pixels: 1223088
16 Average brightness: 128.93
17
18 3. Image Quality Analysis
19 -----
20 Brightness status: Good
21 Sharpness score: 362.07
22 Sharpness status: Sharp
23
24 4. Edge Detection
25 -----
26 Canny edge detection was applied to the input image.
27 Edge map saved to: output/edges.jpg
28
```

Figure 4. Generated Computer Vision Analysis Report

9. TESTING

Testing was performed to verify the correctness and reliability of the major image-processing components of the system. Automated unit tests were implemented using the Pytest framework.

The test suite currently contains three tests covering image property analysis, image quality analysis, and edge detection.

Test Case	Component Tested	Expected Result	Status
TC-01	Image Analysis	Correct image dimensions, channels, and pixel count are returned	Pass
TC-02	Quality Analysis	Brightness and sharpness results are returned correctly	Pass
TC-03	Edge Detection	A valid edge image with the expected dimensions is generated	Pass

The automated test suite was executed using the command `python -m pytest`. All three test cases passed successfully, with the test execution reporting **3 passed in 0.43 seconds**.

In addition to unit testing, the complete application workflow was manually verified by executing the main program with a sample image. The application successfully completed face detection, image analysis, quality assessment, edge detection, output generation, and report generation.

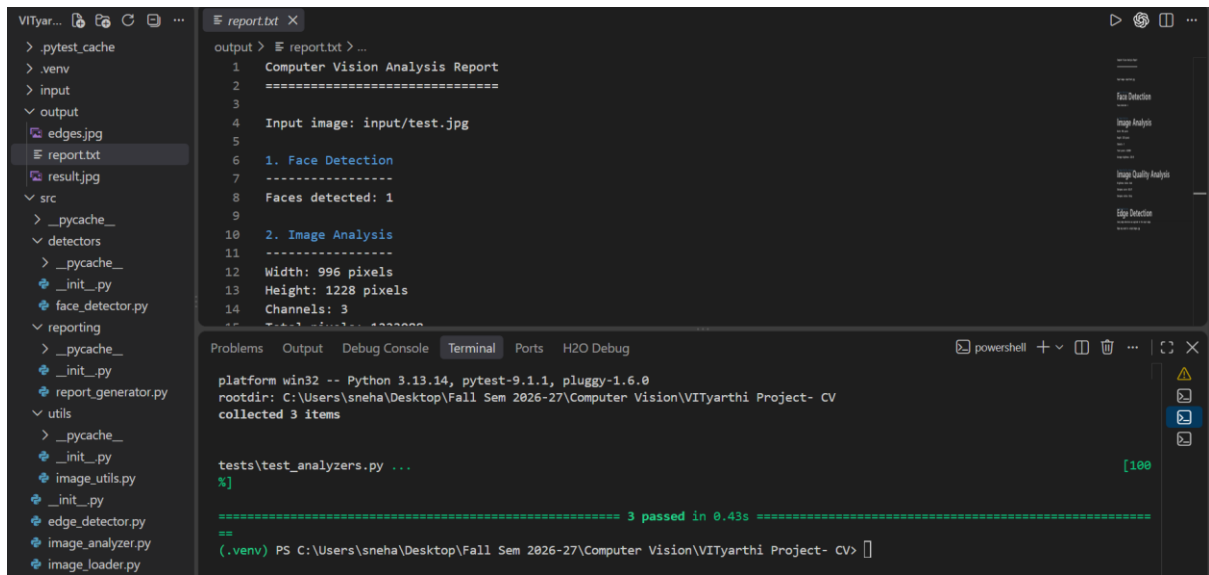


Figure 5. Automated test execution using Pytest

10. CHALLENGES AND LEARNINGS

Challenges

One of the main challenges was integrating multiple computer vision operations into a single reliable workflow. Each processing stage had to produce an appropriate output for the next stage while maintaining a clear separation of responsibilities.

Another challenge was handling invalid image inputs and ensuring that the application produced meaningful error messages instead of failing unexpectedly. Selecting appropriate processing steps and thresholds for brightness, sharpness, and edge detection also required consideration of practical image-processing behavior.

Learnings

The project provided practical experience with fundamental computer vision operations, including grayscale conversion, image filtering, Haar Cascade-based face detection, Canny edge detection, and image-quality measurements.

The project also demonstrated the importance of modular software design, input validation, automated testing, error handling, documentation, and version control when developing a complete software application.

11. FUTURE ENHANCEMENTS

Although the current system implements the core image-analysis workflow successfully, several enhancements could be introduced in future versions:

1. Integration of modern deep-learning-based object detection models for detecting multiple object categories.
2. Addition of a graphical user interface for easier interaction.
3. Support for batch processing of multiple images.
4. Generation of more detailed visual analytics and graphical reports.
5. Addition of configurable detection and image-quality thresholds.
6. Support for additional image formats and advanced image-processing operations.
7. Integration of additional computer vision techniques such as image segmentation and feature extraction.

These enhancements could improve the system's usability, scalability, and analytical capabilities.

12. CONCLUSION

The Computer Vision Image Analyzer successfully integrates multiple fundamental computer vision and image-processing operations into a modular application. The system accepts and validates an input image, performs face detection, extracts basic image properties, evaluates brightness and sharpness, and generates an edge-detection image.

The application also demonstrates practical software engineering practices through modular implementation, input validation, error handling, automated testing, structured output generation, and Git-based version control.

Testing confirmed that the implemented analysis modules function correctly, with all three automated test cases passing successfully. The project therefore demonstrates how fundamental computer vision techniques can be combined into a complete and maintainable image-analysis workflow.

The developed system provides a foundation that can be extended in future versions through deep-learning-based detection, graphical interfaces, batch processing, and additional computer vision techniques.

13. REFERENCES

- OpenCV, *OpenCV Documentation*, Open-Source Computer Vision Library.
- Python Software Foundation, *Python 3 Documentation*, Python Documentation.
- NumPy Developers, *NumPy Documentation*, NumPy Project.
- pytest Development Team, *pytest Documentation*, pytest Documentation.
- Viola, P., & Jones, M. J., “Rapid Object Detection using a Boosted Cascade of Simple Features,” *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, 2001.
- Canny, J., “A Computational Approach to Edge Detection,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-8, no. 6, pp. 679–698, 1986.